

6교시: 컨테이너 실행 검증

수업 목표

- build한 image에서 실행한 container가 실제로 소스 파일을 제공하는지 검증한다.
- docker ps , curl , docker logs , docker exec 를 이용해 port, HTTP, log, 내부 파일 evidence를 확보한다.
- 확인 후 container를 정리한다.

50분 흐름

시간	활동	비중	학생 산출
0-5분	검증 목표 확인	설명 10%	evidence plan
5-15분	container 실행/ps 확인	실행 20%	ps output
15-25분	HTTP와 source text 확인	실행 25%	curl output
25-35분	logs와 내부 파일 확인	실행 25%	log/exec evidence
35-45분	cleanup/recheck	실행 15%	cleanup evidence
45-50분	다음 tag 수업 연결	설명 5%	tag question

Visual 1: 컨테이너 실행 검증

Week 2 Day 2
Lesson 6

실행 검증

Docker build 이후
컨테이너 실행 검증

실행 80%

80%

1 단계 확인 (명령 순서)

- 실행된 컨테이너 확인**
`$ docker ps`
포트 매핑 및 상태 확인
- HTTP 응답 확인**
`$ curl -I http://localhost:18082`
정상 응답(HTTP 200) 확인
- 컨테이너 로그 확인**
`$ docker logs <컨테이너ID>`
nginx 시작 로그 확인
- 내부 파일 확인**
`$ docker exec -it <컨테이너ID> sh`
`# ls -lah /usr/share/nginx/html`
정적 파일 존재 확인
- 정리 (중지/삭제)**
`$ docker stop <컨테이너ID>`
`$ docker rm <컨테이너ID>`
깨끗한 상태로 복귀

실행 중인 컨테이너



이미지 my-nginx:static
컨테이너 이름 web-static
상태 Up (running)
포트 매핑 18082:80

nginx 정적 사이트 제공

포트 매핑 흐름

호스트(내 컴퓨터) → http://localhost:18082 (요청) → 컨테이너(nginx)

호스트 포트 18082 → 컨테이너 포트 80

정상 기준 (체크 포인트)

- ✓ 컨테이너가 Up 상태이고 포트 매핑이 18082:80 임
- ✓ HTTP 요청 시 HTTP 200 응답이 반환됨
- ✓ 로그에 nginx start 완료 메시지가 확인, 내부 정적 파일 존재

증거 (예시 출력)

- docker ps**

CONTAINER ID	IMAGE	STATUS	PORTS	NAMES
a1b2c3d4e5f6	my-nginx:static	Up 3s	0.0.0.0:18082->80/tcp	web-static
- curl -I http://localhost:18082**

```
HTTP/1.1 200 OK
Server: nginx/1.25.4
Date: Sat, 18 May 2024 10:15:30 GMT
Content-Type: text/html
Content-Length: 1256
Connection: keep-alive
Etag: "664c2d18-4e8"
Accept-Ranges: bytes
```

HTTP 200 OK
- docker logs web-static**

```
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to configure
/docker-entrypoint.sh: Configuration complete; ready for start up
2024/05/18 10:15:27 [notice] 1#1: using the "epoll" event method
2024/05/18 10:15:27 [notice] 1#1: nginx/1.25.4
2024/05/18 10:15:27 [notice] 1#1: start worker processes
2024/05/18 10:15:27 [notice] 1#1: start worker process 29
```
- docker exec -it web-static sh**

```
/ # ls -lah /usr/share/nginx/html
total 16K
drwxr-xr-x  2 root root  4.0K May 18 10:15 .
drwxr-xr-x  3 root root  4.0K May 18 10:15 ..
-rw-r--r--  1 root root  1.2K May 18 10:15 index.html
-rw-r--r--  1 root root  1.1K May 18 10:15 style.css
```

5 정리 (깨끗한 상태 복귀)

1 stop (중지)

`$ docker stop web-static`
✓ 컨테이너 중지됨

2 rm (삭제)

`$ docker rm web-static`
✓ 컨테이너 삭제됨

깨끗한 환경 확인

`$ docker ps`
✓ 실행 중인 컨테이너 없음

증거 저장

위 출력 결과를 캡처하여 증거로 저장하세요.

이 이미지는 실행 검증을 ps , HTTP, logs , exec , cleanup으로 나눈다. 하나만 성공해도 충분하다고 보지 않고 서로 다른 증거를 모아 정상 상태를 확정한다.

Hands-on

```
docker run -d --name paperclip-day2-static -p 18082:80 paperclip-static-site:day2
docker ps --filter name=paperclip-day2-static
```

```
curl -I http://localhost:18082
curl -s http://localhost:18082 | grep "Dockerfile로 만든 표준 실습 앱"
docker logs paperclip-day2-static
docker exec paperclip-day2-static ls -l /usr/share/nginx/html
docker stop paperclip-day2-static
docker rm paperclip-day2-static
```

Linux 사전 테스트 결과

내부 파일 확인:

```
total 12
-rw-r--r-- 1 root root 497 50x.html
-rwxrwxrwx 1 root root 1188 index.html
-rwxrwxrwx 1 root root 1156 styles.css
```

source text 확인:

```
<h1>Dockerfile로 만든 표준 실습 앱</h1>
```

다중 evidence가 필요한 이유

하나의 evidence만으로는 운영 판단이 약할 수 있다. `docker ps` 에서 container가 Up 이어도 HTTP가 실패할 수 있고, HTTP가 200이어도 기대한 source가 image에 들어갔는지는 확인되지 않을 수 있다. 그래서 Day 2는 최소 네 종류의 evidence를 모은다.

evidence	확인하는 층	놓칠 수 있는 것
<code>docker ps</code>	container lifecycle, port binding	앱 내용이 맞는지
<code>curl -I</code>	HTTP status/header	body 내용이 맞는지
<code>curl -s ... grep</code>	source content	내부 path 구조
<code>docker logs</code>	process log/request log	image에 파일이 있는지
<code>docker exec ls</code>	container filesystem	외부 접속 가능 여부

이렇게 층을 나누어 보면 장애 분석도 쉬워진다. `ps` 는 정상인데 `curl` 이 실패하면 port/network 쪽을 본다. `curl -I` 는 200인데 body가 다르면 build context나 COPY 결과를 본다. `exec` 에서 파일이 없으면 Dockerfile의 `WORKDIR` / `COPY` 를 다시 본다.

운영 handoff 기준

다른 엔지니어에게 넘길 때는 "실행됩니다"가 아니라 다음 정보가 필요하다.

```
image: paperclip-static-site:day2
container: paperclip-day2-static
host port: 18082
container port: 80
expected status: HTTP/1.1 200 OK
expected text: Dockerfile로 만든 표준 실습 앱
cleanup: docker stop + docker rm
```

핵심 유의사항

6교시는 검증 중심 수업이다. `curl` 한 번 성공했다고 끝내면 evidence가 부족하다. container lifecycle, network/port, HTTP header, body content, 내부 filesystem은 서로 다른 층의 evidence다. 한 층이 정상이어도 다른 층이 틀릴 수 있다.

`docker logs` 는 application log이면서 `nginx access log` 역할도 한다. `curl` 을 실행한 뒤 `logs`를 보면 요청 흔적이 남는다. 이 연결을 보여주면 학생은 "내가 보낸 요청이 container 안 process까지 도달했는가"를 확인하는 감각을 갖게 된다.

`docker exec` 는 container 안으로 들어가서 수정하라는 명령이 아니다. 초급 과정에서는 내부 상태 확인 도구로 사용한다. container 안 파일을 직접 수정하면 image build와 runtime writable layer를 혼동하게 되므로, 수정은 host source를 바꾸고 rebuild하거나 bind mount 실습에서 다룬다.

cleanup은 선택이 아니다. 실습 container가 남아 있으면 다음 학생이나 다음 교시에서 port 충돌이 생긴다. 특히 7교시 bind mount 실습은 같은 `nginx port`를 다시 쓰므로 6교시 끝에서 `docker ps` 재확인까지 해야 한다.

자주 놓치는 검증 지점

놓치는 지점	겉으로 보이는 증상	바로잡는 확인
<code>docker ps</code> 만 보고 성공 처리	HTTP 접속 실패를 늦게 발견	<code>curl -I</code> 까지 실행
header만 확인	예전 image의 body를 보고 있음	<code>body text grep</code>
<code>logs</code> 를 안 봄	요청이 container까지 갔는지 모름	<code>docker logs</code>
<code>exec</code> 를 수정 도구로 사용	image와 writable layer 혼동	<code>exec ls</code> 처럼 read-only 확인
cleanup 생략	다음 실습 port/name 충돌	<code>docker ps --filter name=paperclip</code>

장애 분기표

관찰 결과	가능성	다음 명령
<code>docker ps</code> 에 container 없음	process가 종료됨	<code>docker ps -a</code> , <code>docker logs</code>
container는 Up, <code>curl</code> 실패	port publish 문제	<code>docker ps</code> PORTS 확인
<code>curl -I 200</code> , body text 없음	다른 image/tag 실행 가능성	<code>docker images</code> , run command 확인
body는 맞지만 <code>logs</code> 없음	요청 방식 또는 log 확인 시점 문제	<code>curl</code> 재실행 후 <code>docker logs</code>
<code>exec</code> 에서 파일 없음	WORKDIR/COPY 문제	<code>docker history</code> , Dockerfile 확인

학생 기록 템플릿

```
## Lesson 6 Runtime Verification Evidence
- image:
- container name:
- `docker ps` status:
- PORTS:
- HTTP status:
- body text check:
- logs에서 확인한 요청:
- 내부 파일 목록:
- cleanup 후 `docker ps` 결과:
```

빠른 학생 확장

시간이 남으면 같은 image를 다른 container name과 host port로 실행해 본다.

```
docker run -d --name paperclip-day2-static-alt -p 18084:80 paperclip-static-site:day2
curl -I http://localhost:18084
```

```
docker logs paperclip-day2-static-alt
docker rm -f paperclip-day2-static-alt
```

이 확장은 image와 container instance의 차이를 다시 확인한다. image는 같지만 container name과 host port는 다를 수 있다. 반대로 같은 container name은 동시에 두 번 사용할 수 없다.

cleanup audit

마무리 단계에서 아래 명령을 실행한다.

```
docker ps --filter name=paperclip
docker ps -a --filter name=paperclip
```

정상 기준은 실행 중인 Day 2 container가 없고, 다음 실습에서 사용할 name과 port가 비어 있는 것이다. 만약 남아 있다면 이름을 확인하고 수업용 container인지 판단한 뒤 정리한다.

평가 기준

기준	2점 evidence
ps	container status와 port binding을 확인했다.
HTTP	curl -I와 source text를 확인했다.
logs	nginx startup 또는 request log를 찾았다.
exec	container 내부 파일 위치를 확인했다.
cleanup	stop/rm 후 남은 container가 없음을 확인했다.

공식 근거 링크

- Docker exec: <https://docs.docker.com/reference/cli/docker/container/exec/>
- Docker logs: <https://docs.docker.com/reference/cli/docker/container/logs/>
- Docker run: <https://docs.docker.com/reference/cli/docker/container/run/>